3 Writing a Migration

Once you have created your migration using one of the generators it's time to get to work!

3.1 Creating a Table

The `create_table` method is one of the most fundamental, but most of the time, will be generated for you from using a model or scaffold generator. A typical use would be

```
create_table :products do |t|
  t.string :name
end
```

which creates a `products` table with a column called `name` (and as discussed below, an implicit `id`column).

By default, `create_table` will create a primary key called `id`. You can change the name of the primary key with the `:primary_key` option (don't forget to update the corresponding model) or, if you don't want a primary key at all, you can pass the option `id: false`. If you need to pass database specific options you can place an SQL fragment in the `:options` option. For example:

```
create_table :products, options: "ENGINE=BLACKHOLE" do |t|
  t.string :name, null: false
end
```

will append `ENGINE=BLACKHOLE` to the SQL statement used to create the table (when using MySQL, the default is `ENGINE=InnoDB`).

3.2 Creating a Join Table

Migration method `create_join_table` creates a HABTM join table. A typical use would be:

```
create_join_table :products, :categories
```

which creates a `categories_products` table with two columns called `category_id` and `product_id`. These columns have the option `:null` set to `false` by default. This can be overridden by specifying the `:column_options` option.

```
create_join_table :products, :categories, column_options: {null: true}
```

will create the `product_id` and `category_id` with the `:null` option as `true`.

You can pass the option `:table_name` when you want to customize the table name. For example:

```
create_join_table :products, :categories, table_name: :categorization
```

will create a `categorization` table.

`create_join_table` also accepts a block, which you can use to add indices (which are not created by default) or additional columns:

```
create_join_table :products, :categories do |t|
  t.index :product_id
  t.index :category_id
end
```

3.3 Changing Tables

A close cousin of `create_table` is `change_table`, used for changing existing tables. It is used in a similar fashion to `create_table` but the object yielded to the block knows more tricks. For example:

```
change_table :products do |t|
  t.remove :description, :name
  t.string :part_number
  t.index :part_number
  t.rename :upccode, :upc_code
end
```

removes the `description` and `name` columns, creates a `part_number` string column and adds an index on it. Finally it renames the `upccode` column.